

Random Numbers, Simulation, and Interpolation

MATLAB Programming

Jin Hee Yoon

Sejong University

Spring, 2026

이번 강의에서는 MATLAB을 이용하여 난수 생성, 시뮬레이션, 보간법을 하나의 흐름으로 학습한다.

- 난수를 이용하여 인공 데이터를 생성하는 방법을 이해한다.
- 반복 실험을 통해 확률적 현상을 시뮬레이션한다.
- Monte Carlo 방법을 이용하여 수치적 근사를 수행한다.
- 불완전한 데이터에서 보간법을 이용하여 값을 복원한다.
- 난수, 시뮬레이션, 보간법을 결합한 응용 문제를 해결한다.

Simulation은 실제 시스템을 직접 실험하기 어렵거나 비용이 큰 경우, 그 시스템의 동작을 수학적 모델과 컴퓨터 계산으로 모방하여 분석하는 방법이다.

- 실제 실험이 위험하거나 비용이 큰 경우
예) 전력망 고장, 반도체 회로 오류, 자율주행 사고 상황
- 시스템의 미래 동작을 예측하고 싶은 경우
예) 통신망 트래픽, 배터리 수명, 서버 부하
- 다양한 조건을 반복적으로 시험하는 경우
예) 회로 파라미터 변화, 알고리즘 성능 비교

Why Random Numbers in Simulation ?

현실의 시스템에는 항상 불확실성, 잡음, 변동성이 존재하며, 난수는 이를 컴퓨터에서 모사하기 위한 핵심 도구이다.

- 인공지능
예) 데이터 샘플링, 모델 초기화, 확률적 학습
- 전기/전자
예) 회로 잡음, 센서 오차, 고장 확률 분석
- 컴퓨터 시스템
예) 네트워크 트래픽, 큐잉 시스템, 보안 알고리즘

Monte Carlo Simulation

Monte Carlo Simulation은 난수를 이용하여 동일한 실험을 반복 수행하고, 그 결과의 통계적 성질을 통해 문제의 해를 근사적으로 추정하는 방법이다.

- 해석적으로 풀기 어려운 문제를 해결하기 위한 방법
- 확률적 입력을 생성하여 시스템을 반복적으로 실행
- 결과의 평균, 분산, 분포 등을 이용하여 해를 추정

Monte Carlo 시뮬레이션의 특징

- Random Sampling: 난수를 이용한 입력 생성
- Repetition: 동일 실험의 반복 수행
- Statistical Estimation: 평균, 확률, 분포 추정

Why is it called Monte Carlo?

Monte Carlo라는 이름은 수학적 개념이 아니라, 난수 기반의 확률적 반복 실험이 도박 (gambling)과 유사하다는 점에서 유래하였다.

- 난수를 이용한 실험 → 결과가 확률적으로 결정됨
- 동일한 실험을 반복 → 통계적으로 결과를 추정
- 이러한 특징이 카지노 게임과 매우 유사

Historical Background

- Stanislaw Ulam이 카드 게임(솔리테어) 확률 문제를 고민하다가 아이디어 착안
- John von Neumann과 함께 난수 기반 계산 방법으로 발전
- 카지노로 유명한 Monte Carlo (Monaco)의 이름을 따서 명명

난수, 시뮬레이션, 보간법은 서로 독립된 주제가 아니라 데이터 분석 과정에서 자연스럽게 연결된다.

- **Random number:** 현실의 불확실성을 수치로 표현한다.
- **Simulation:** 난수를 반복적으로 발생시켜 현상을 실험한다.
- **Interpolation:** 관측되지 않은 중간 값을 추정한다.

Random numbers $\longrightarrow$ Simulation $\longrightarrow$ Interpolation

Random Numbers

난수는 단순히 임의의 숫자가 아니라 특정한 확률분포에서 생성된 값이다.

- MATLAB의 난수 함수는 확률적 실험을 컴퓨터에서 구현하는 기본 도구이다.
- 같은 분포라도 매번 다른 표본이 생성될 수 있다.
- 시뮬레이션에서는 많은 난수를 생성하여 평균적 행동을 관찰한다.

예를 들어 동전 던지기, 주사위 던지기, 측정 오차, 센서 잡음 등은 모두 난수로 표현할 수 있다.

MATLAB의 주요 난수 함수

MATLAB에서는 다양한 확률분포에서 난수를 생성할 수 있다.

함수	사용 문법	의미
rand	rand(m,n)	$U(0, 1)$ 균등분포 난수
randn	randn(m,n)	$N(0, 1)$ 정규분포 난수
randi	randi([a,b],m,n)	[a,b] 구간 정수 난수 생성
rng	rng(seed)	난수 seed 설정
randperm	randperm(n)	$1 \sim n$ 무작위 순열
unifrnd	unifrnd(a,b,m,n)	$U(a, b)$ 균등분포
normrnd	normrnd(μ, σ, m, n)	$N(\mu, \sigma)$ 정규분포
exprnd	exprnd(λ, m, n)	지수분포 난수
poissrnd	poissrnd(λ, m, n)	포아송 분포 난수

- (m, n): 생성할 난수의 크기 (행렬크기)
- λ : 평균(지수/포아송 분포)

난수 함수 선택 기준

난수 함수는 문제의 특성에 맞게 선택해야 한다.

- **균등분포**: 모든 값이 동일한 확률 → 기본 sampling
- **정규분포**: 자연현상, 노이즈 모델링
- **지수분포**: 사건 사이의 시간 간격
- **포아송분포**: 일정 시간 동안 발생 횟수

예:

- 센서 노이즈 → `randn`
- 고객 도착 시간 → `exprnd`
- 전화 수 → `poissrnd`

난수 선택은 모델링의 핵심이다.

다양한 난수 생성 예제

```
clear; clc;
u = rand(1,1000);           % Uniform(0,1)
n = randn(1,1000);          % Standard normal
d = randi([1 6],1,1000);    % Dice simulation
p = randperm(10);           % Random permutation of 1~10
u2 = 5 + (10-5)*rand(1,1000); % Uniform(5,10)
n2 = 50 + 5*randn(1,1000);   % Normal with mean=50, std=5
e = exprnd(2,1,1000);        % Exponential (mean=2)
po = poissrnd(3,1,1000);     % Poisson (lambda=3)
subplot(2,2,1); histogram(u); title('Uniform');
subplot(2,2,2); histogram(n); title('Normal');
subplot(2,2,3); histogram(e); title('Exponential');
subplot(2,2,4); histogram(po); title('Poisson');
```

Uniform Random Numbers: rand

```
clear; clc; % Clear workspace and command window
x = rand(1,1000); % Generate 1000 uniform random numbers
histogram(x); % Draw histogram
title('Uniform Random Numbers'); % Add title
xlabel('Value'); % Add x-axis label
ylabel('Frequency'); % Add y-axis label
```

rand는 0과 1 사이의 값을 거의 동일한 가능성으로 생성한다.

Uniform Distribution의 의미

균등분포는 모든 구간이 같은 가능성을 갖는 분포이다.

$$X \sim U(0, 1)$$

- 0에 가까운 값과 1에 가까운 값이 특별히 더 자주 나오지 않는다.
- Monte Carlo simulation의 기본 재료로 자주 사용된다.
- 다른 분포를 만드는 출발점으로도 사용된다.

Normal Random Numbers: randn

```
clear; clc; % Clear workspace and command window
x = randn(1,1000); % Generate 1000 standard normal numbers
histogram(x); % Draw histogram
title('Normal Random Numbers'); % Add title
xlabel('Value'); % Add x-axis label
ylabel('Frequency'); % Add y-axis label
```

randn은 평균 0, 표준편차 1인 표준정규분포 난수를 생성한다.

정규분포는 자연현상과 측정 오차를 모델링할 때 매우 자주 사용된다. 특히 평균 0, 표준편차 1인 표준정규분포는 Monte Carlo 시뮬레이션에서 자주 사용된다.

$$X \sim N(0, 1)$$

- 평균 주변의 값이 자주 발생한다.
- 평균에서 멀어질수록 발생 빈도가 낮아진다.
- 센서 잡음, 측정 오차, 자연적 변동성을 표현할 때 유용하다.

Mean과 Standard Deviation을 갖는 정규분포

```
clear; clc; % Clear workspace and command window
mu = 70; % Set mean
sigma = 10; % Set standard deviation
x = mu + sigma*randn(1,1000); % Generate normal random numbers
histogram(x); % Draw histogram
title('Normal Random Numbers'); % Add title
xlabel('Value'); % Add x-axis label
ylabel('Frequency'); % Add y-axis label
```

표준정규분포 난수에 표준편차를 곱하고 평균을 더하면 원하는 평균과 표준편차를 갖는 난수를 만들 수 있다.

Integer Random Numbers: randi

```
clear; clc;                                % Clear workspace and command window
dice = randi([1 6],1,1000);                % Simulate 1000 dice rolls
histogram(dice);                            % Draw histogram
title('Dice Simulation');                  % Add title
xlabel('Dice Value');                      % Add x-axis label
ylabel('Frequency');                      % Add y-axis label
```

`randi([1 6])`는 주사위처럼 1부터 6까지의 정수 난수를 생성한다.

난수는 실행할 때마다 달라질 수 있다. 그러나 실험 결과를 다시 확인하거나 과제 채점을 할 때는 같은 결과가 재현되어야 한다.

- rng는 난수 생성의 시작점을 고정한다.
- 같은 seed를 사용하면 같은 난수열이 생성된다.
- 연구, 과제, 디버깅에서 재현성은 매우 중요하다.

Using rng

```
clear; clc; % Clear workspace and command window
rng(1); % Fix random seed
a = rand(1,5); % Generate 5 random numbers
rng(1); % Use the same seed again
b = rand(1,5); % Generate the same 5 random numbers
disp(a); % Display first result
disp(b); % Display second result
```

같은 seed를 사용했기 때문에 a와 b는 동일한 값이 된다.

난수를 많이 생성하면 표본 평균($\bar{X}$)은 이론적 평균(μ)에 가까워진다. (Central Limit Theorem, 중심극한정리)

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i \quad \longrightarrow \quad \mu \quad \text{if} \quad n \quad \longrightarrow \quad \infty$$

- 표본 수가 작으면 평균이 많이 흔들릴 수 있다.
- 표본 수가 커지면 평균이 안정된다.
- 이것이 시뮬레이션을 많이 반복하는 이유이다.

Law of Large Numbers (Effect of n)

Example

동전 던지기 실험에서 시행 횟수 n 이 증가할수록 앞면의 비율이 이론적 확률 0.5에 수렴하는 과정을 확인한다.

- 작은 n 에서는 결과의 변동이 매우 큼
- n 이 증가할수록 평균이 점점 안정됨
- 서로 다른 n 에서 수렴 속도를 비교 가능

Idea

$$\text{Running Mean} = \frac{X_1 + X_2 + \cdots + X_n}{n}$$

$n \rightarrow \infty$ 이면 이 값은 0.5로 수렴

Law of Large Numbers (MATLAB Simulation)

```
clear; clc;
N_list = [50, 200, 1000, 10000]; % Different sample sizes
figure; hold on;
for k = 1:length(N_list)
    n = N_list(k);
    coin = rand(1,n) > 0.5; % Coin toss
    running_mean = cumsum(coin)./(1:n); % Running mean

    plot(running_mean, 'LineWidth', 1.5);
end
yline(0.5,'k--','LineWidth',2); % True probability
legend('n=50','n=200','n=1000','n=10000');
xlabel('Number of Trials');
ylabel('Proportion of Heads');
grid on;
```

Simulation

시뮬레이션은 현실 현상이나 수학적 문제를 컴퓨터 실험으로 재현하는 방법이다.

- 직접 실험하기 어렵거나 비용이 큰 문제를 컴퓨터로 실험할 수 있다.
- 확률적 요소가 있는 문제는 난수를 이용하여 구현한다.
- 반복 횟수를 늘리면 평균적 행동을 더 안정적으로 관찰할 수 있다.

Monte Carlo simulation은 난수를 반복적으로 사용하여 수치적 결과를 근사하는 방법이다.

- 확률, 적분, 기대값 계산에 자주 사용된다.
- 해석적으로 풀기 어려운 문제도 실험적으로 접근할 수 있다.
- 정확도는 반복 횟수에 크게 영향을 받는다.

Many random trials $\Rightarrow$ Approximate numerical answer

Monte Carlo: π 추정

Example

단위 정사각형 $[0, 1] \times [0, 1]$ 안에 임의의 점을 생성하고, 원점 중심 반지름 1인 사분원의 내부에 포함되는 점의 비율을 이용하여 π 를 추정한다.

- 사분원 면적 = $\frac{\pi}{4}$
- 내부 점 비율 $\approx \frac{\pi}{4}$
- 따라서 $\pi \approx 4 \times (\text{inside ratio})$

```
clear; clc; % Clear workspace and command window
n = 10000; % Set number of random points
x = rand(1,n); % Generate x-coordinates
y = rand(1,n); % Generate y-coordinates
inside = (x.^2 + y.^2) <= 1; % Check points inside quarter circle
pi_est = 4*mean(inside); % Estimate pi
fprintf('pi estimate = %.4f\n',pi_est); % Print result
scatter(x,y,5,inside,'filled'); % Plot random points
axis equal; % Use equal axis scale
```

단위 정사각형 $[0, 1] \times [0, 1]$ 안에 점을 무작위로 찍는다고 하자.

$$x^2 + y^2 \leq 1$$

을 만족하는 점은 반지름이 1인 사분원 안에 들어간다.

$$\frac{\text{사분원 넓이}}{\text{정사각형 넓이}} = \frac{\pi/4}{1} = \frac{\pi}{4}$$

따라서 사분원 안에 들어간 점의 비율에 4를 곱하면 π 를 근사할 수 있다.

반복 횟수에 따른 π 추정

```
clear; clc; % Clear workspace and command window
n_list = [100 500 1000 5000 10000]; % Set different sample sizes
pi_values = zeros(size(n_list)); % Preallocate result vector
for k = 1:length(n_list) % Repeat for each sample size
    n = n_list(k); % Select current sample size
    x = rand(1,n); % Generate x-coordinates
    y = rand(1,n); % Generate y-coordinates
    inside = (x.^2 + y.^2) <= 1; % Check inside quarter circle
    pi_values(k) = 4*mean(inside); % Store pi estimate
end
plot(n_list,pi_values,'o-'); % Plot estimates
yline(pi,'--'); % Add true pi value
xlabel('Number of Points'); % Add x-axis label
ylabel('Estimated pi'); % Add y-axis label
```

Monte Carlo: Probability Estimation

Example

주사위를 여러 번 던져 “합이 7 이상”이 나올 확률을 Monte Carlo로 추정

- 난수를 이용하여 주사위 결과 생성
- 반복 실험을 통해 확률을 근사

```
clc; clear;

n = 10000;           % 실험 횟수
dice1 = randi(6,1,n); % 첫 번째 주사위
dice2 = randi(6,1,n); % 두 번째 주사위

sum_dice = dice1 + dice2; % 합 계산
event = (sum_dice >= 7); % 사건 정의

prob_est = mean(event); % 확률 추정

fprintf('Estimated Probability = %.4f\n', prob_est);
```

Monte Carlo Integration

Example

난수 $X \sim U(0,1)$ 를 생성하고 함수값 $f(X)$ 의 평균을 이용하여 정적분 $\int_0^1 f(x) dx$ 를 근사한다.

- 균등분포에서 샘플 생성
- 함수값의 평균으로 적분값 추정
- 샘플 수가 증가할수록 정확도 향상

```
clear; clc; % Clear workspace and command window
n = 10000; % Set number of samples
x = rand(1,n); % Generate samples from U(0,1)
f = exp(-x.^2); % Evaluate function values
I_est = mean(f); % Estimate integral on [0,1]
fprintf('Integral estimate = %.4f\n', I_est); % Print result
histogram(f); % Visualize function values
```

$X \sim U(0,1)$ 이면

$$\int_0^1 f(x) dx \approx \frac{1}{n} \sum_{i=1}^n f(X_i)$$

함수의 적분을 넓이로 생각하면, Monte Carlo 방법은 무작위 점에서 함수값을 계산하여 평균을 구하는 방식이다.

- 격자점이 아니라 무작위 점을 사용한다.
- 차원이 높아질수록 전통적 격자 방법보다 유리할 수 있다.
- 다만 결과가 난수에 따라 조금씩 변한다.

Monte Carlo: Numerical Integration

Example

$\int_0^1 x^2 dx$ 값을 Monte Carlo로 근사

- 구간 $[0, 1]$ 에서 난수 생성
- 함수값 평균을 이용하여 적분값 추정

```
clc; clear;

n = 10000;           % 샘플 개수
x = rand(1,n);       % [0,1] 난수 생성

y = x.^2;            % 함수값 계산

integral_est = mean(y); % 평균값 = 적분 근사

fprintf('Estimated Integral = %.4f\n', integral_est);
```

Random walk는 매 순간 무작위 방향으로 움직이는 과정을 나타낸다.

- 한 번의 경로만 보면 특정한 방향성이 있는 것처럼 보일 수 있다.
- 그러나 많은 경로를 평균내면 확률적 구조가 드러난다.
- 금융 가격, 입자 이동, 확산 현상 등에서 기본 모델로 사용된다.

Monte Carlo: Random Walk

Example

술에 취한 사람이 왼쪽 또는 오른쪽으로 무작위로 한 걸음씩 이동한다고 가정하고, 시간에 따른 위치 변화를 시뮬레이션한다.

```
clc; clear;

n = 200;                                % Number of steps

step = randi([0,1],1,n);                % Generate 0 or 1
step(step == 0) = -1;                    % Convert 0 to -1

position = cumsum(step);                  % Cumulative position

plot(1:n, position, 'LineWidth', 1.5);
xlabel('Step');
ylabel('Position');
title('Random Walk Simulation');
grid on;
```

Monte Carlo: 여러 개의 Random Walk 경로

Example

각 단계에서 +1 또는 -1로 무작위 이동하는 입자를 여러 개 생성하여, 시간에 따른 다양한 경로의 변화를 시각적으로 확인한다.

- 각 경로는 독립적인 확률적 과정
- 동일한 규칙이지만 결과는 모두 다르게 나타남
- 시간이 지날수록 분산(퍼짐)이 증가

```
clear; clc; % Clear workspace and command window
n = 300; % Set number of steps
m = 20; % Set number of paths
steps = randi([0 1],n,m); % Generate random steps
steps = 2*steps - 1; % Convert 0/1 to -1/+1
walks = cumsum(steps); % Compute cumulative paths
plot(walks); % Plot all random walks
xlabel('Step'); % Add x-axis label
ylabel('Position'); % Add y-axis label
title('Multiple Random Walks'); % Add title
```

Monte Carlo: Waiting Time at a Cafeteria

Example

학생들이 식당에 무작위로 도착하고, 음식 준비 시간도 매번 다르다고 가정하여 평균 대기시간을 추정한다.

```
clc; clear;n = 1000;                                % Number of students
arrival = cumsum(exprnd(2,1,n));    % Random arrival times
service = exprnd(1.5,1,n);         % Random service times
start_time = zeros(1,n); finish_time = zeros(1,n);
for i = 1:n
    if i == 1
        start_time(i) = arrival(i);
    else
        start_time(i) = max(arrival(i), finish_time(i-1));
    end
    finish_time(i) = start_time(i) + service(i);
end
waiting_time = start_time - arrival;
fprintf('Average waiting time = %.3f\n', mean(waiting_time));
histogram(waiting_time, 30);xlabel('Waiting Time');ylabel('Frequency');
title('Waiting Time Distribution');
```

Monte Carlo: Sensor Noise Simulation

Example

실제 온도가 일정하더라도 센서에는 잡음이 포함된다. 난수를 이용하여 센서 측정값의 변동성을 시뮬레이션한다.

```
clc; clear;n = 200;                                % Number of measurements
true_temp = 25;                                     % True temperature
noise = 0.8*randn(1,n);                             % Measurement noise
measured_temp = true_temp + noise;
plot(1:n, measured_temp, 'o-');
hold on;
yline(true_temp, 'LineWidth', 2);
xlabel('Measurement Index');
ylabel('Temperature');
title('Sensor Noise Simulation');
grid on;
```

Monte Carlo: System Reliability

Example

3개의 부품이 모두 정상일 때만 작동하는 시스템의 전체 신뢰도(시스템작동률)를 시뮬레이션으로 추정해보자.

```
clc; clear; n = 10000;                                % Number of simulations
p1 = 0.98;                                              % Reliability of component 1
p2 = 0.95;                                              % Reliability of component 2
p3 = 0.97;                                              % Reliability of component 3
c1 = rand(1,n) < p1;
c2 = rand(1,n) < p2;
c3 = rand(1,n) < p3;
system_working = c1 & c2 & c3;
reliability_est = mean(system_working);
fprintf('Estimated system reliability = %.4f\n', reliability_est);
```

Monte Carlo: Battery Lifetime

Example

스마트폰 배터리 사용 시간이 매번 다르다고 가정하고, 하루 사용 중 배터리가 10시간 이상 지속될 확률을 추정하자. (배터리수명은 평균8, 표준편차1.5인 정규분포라 가정)

```
clc; clear; n = 10000; % Number of simulations
battery_life = 8 + 1.5*randn(1,n); % Random battery lifetime
event = battery_life >= 10; % Battery lasts at least 10 hours
prob_est = mean(event);
fprintf('Probability = %.4f\n', prob_est);
histogram(battery_life, 40);
xlabel('Battery Lifetime');
ylabel('Frequency');
title('Battery Lifetime Simulation');
```

$P(\text{battery life} \geq 10)$ 를 구해줌

Monte Carlo: 보험 손실 Risk Simulation

이 예제는 보험 리스크, 고객 손실 시뮬레이션, Monte Carlo simulation 에 관한 예제이다.

Example

보험회사가 10,000명의 고객을 대상으로 손실액을 시뮬레이션한다고 하자. 각 고객은 일정한 확률로 보험금을 청구할 수 있으며, 보험금을 청구한 경우 청구 금액은 평균 500, 표준편차 200인 정규분포를 따른다고 가정하고, 보험청구비율은 8%로 가정하자.

다음 순서로 보험 손실액을 추정해보자.

- 고객별 보험금 청구 여부를 난수로 생성한다.
- 청구가 발생한 고객에 대해서만 손실액을 계산한다.
- 전체 손실액과 고객 1인당 평균 손실액을 추정한다.
- 손실액 분포를 histogram으로 확인한다.

Monte Carlo: 보험 손실 Risk Simulation

```
clear; clc; n = 10000; % Number of customers

claim_prob = 0.08; % Probability of claim
claim_amount = 500 + 200*randn(1,n); % Claim amount (Normal)
has_claim = rand(1,n) < claim_prob; % Claim occurrence (Bernoulli)
loss = has_claim .* claim_amount; % Final loss per customer
total_loss = sum(loss); % Total loss
mean_loss = mean(loss); % Mean loss

fprintf('Total loss = %.2f\n',total_loss); % Print total loss
fprintf('Mean loss = %.2f\n',mean_loss); % Print mean loss

histogram(loss); % Plot loss distribution
title('Simulated Insurance Loss'); % Title
xlabel('Loss Amount'); % X label
ylabel('Frequency'); % Y label
```

이 예제는 보험금 청구 여부와 청구 금액을 난수로 모델링한 것이다.

- 청구 여부는 Bernoulli trial로 생각할 수 있다.
- 청구 금액은 정규분포를 이용하여 생성하였다.
- 전체 손실액은 개별 손실액의 합으로 계산된다.

이처럼 시뮬레이션은 현실적 의사결정 문제에도 직접 연결된다.

Interpolation

보간법은 이미 알고 있는 데이터 점들 사이의 값을 추정하는 방법이다.

- 모든 데이터를 촘촘하게 관측하기 어려운 경우가 많다.
- 센서 데이터, 실험 데이터, 시간 변화 데이터에서 중간 값 추정이 필요하다.
- MATLAB에서는 `interp1`, `interp2` 등을 사용한다.

$$(x_1, y_1), \dots, (x_n, y_n) \Rightarrow y(x^*) \text{ 추정}$$

보간법에서는 주어진 데이터 점을 이용하여 새로운 위치에서의 값을 추정한다.

- `interp1`: 1차원 보간 (x-y 데이터)
- `interp2`: 2차원 보간 (격자 형태 데이터)

Interpolation과 Extrapolation

- **Interpolation:** 관측된 데이터 범위 안에서 값을 추정한다.
- **Extrapolation:** 관측된 데이터 범위 밖에서 값을 추정한다.

$$x^* \in [x_{\min}, x_{\max}] \Rightarrow \text{interpolation}$$

$$x^* < x_{\min} \text{ or } x^* > x_{\max} \Rightarrow \text{extrapolation}$$

Extrapolation은 데이터가 없는 영역을 예측하는 것이므로 훨씬 더 조심해야 한다.

Linear Interpolation: 1차원보간 interp1

```
clear; clc;                                % Clear workspace and command window
x = 1:5;                                    % Define observed x-values
y = [2 4 3 5 7];                           % Define observed y-values
xq = 1:0.1:5;                               % Define query points
yq = interp1(x,y,xq,'linear');              % Perform linear interpolation
plot(x,y,'o',xq,yq,'LineWidth',1.5);       % Plot data and interpolation
xlabel('x');                                % Add x-axis label
ylabel('y');                                % Add y-axis label
legend('Data','Linear');                    % Add legend
```

Linear Interpolation의 특징

Linear interpolation은 두 관측점 사이를 직선으로 연결한다.

- 계산이 간단하고 해석이 쉽다.
- 데이터가 거의 선형적으로 변할 때 적절하다.
- 곡률이 큰 함수에서는 부드럽지 않게 보일 수 있다.

두 점 (x_1, y_1) , (x_2, y_2) 사이에서 보간값은 직선 방정식으로 계산된다.

Linear Interpolation: 2차원 보간interp2 예제

Example

격자(grid) 형태의 데이터가 주어졌을 때, 더 촘촘한 격자에서의 값을 추정한다.

예를 들어 함수 $z = x^2 + y^2$ 를 일부 점에서만 알고 있을 때, 중간 위치의 값을 보간으로 계산할 수 있다.

```
clear; clc;
[X,Y] = meshgrid(1:5,1:5);           % Create coarse grid
Z = X.^2 + Y.^2;                     % Define surface function
[Xq,Yq] = meshgrid(1:0.2:5,1:0.2:5); % Create fine grid
Zq = interp2(X,Y,Z,Xq,Yq,'linear'); % 2D interpolation
subplot(1,2,1)
surf(X,Y,Z)                          % Original coarse surface
title('Original Grid')
subplot(1,2,2)
surf(Xq,Yq,Zq)                      % Interpolated surface
title('Interpolated Surface')
xlabel('X'); ylabel('Y'); zlabel('Z');
```

- 원래 데이터는 거친 격자(coarse grid)에서만 존재. 보간을 통해 더 부드러운 surface를 생성

3차원 보간: interp3 예제

Example

3차원 격자 데이터가 주어졌을 때, 더 촘촘한 공간에서의 값을 추정한다. 예를 들어 함수

$$v = x^2 + y^2 + z^2$$

를 일부 격자점에서만 알고 있을 때, 보간을 통해 내부 공간의 값을 추정할 수 있다.

```
clear; clc; % Clear workspace
[X,Y,Z] = meshgrid(1:4,1:4,1:4); % Create coarse 3D grid
V = X.^2 + Y.^2 + Z.^2; % Define 3D function
[Xq,Yq,Zq] = meshgrid(1:0.5:4,1:0.5:4,1:0.5:4); % Fine grid
Vq = interp3(X,Y,Z,V,Xq,Yq,Zq,'linear'); % 3D interpolation
% Slice visualization
slice(Xq,Yq,Zq,Vq,2,2,2); % Cross-sectional slices
shading interp; % Smooth shading
colorbar; % Show color scale
xlabel('X'); ylabel('Y'); zlabel('Z');
title('3D Interpolated Volume');
```

Spline Interpolation의 특징

Spline interpolation은 여러 구간에서 3차 다항식을 이용하여 부드러운 곡선을 이어 붙이는 방법이다. (곡선보간에서는 주로 3차 다항식이용, 4차 이상 가능하나 과적합 또는 불안정)

- Linear interpolation보다 부드러운 곡선을 만든다.
- 함수가 부드럽게 변할 때 좋은 결과를 낼 수 있다.
- 데이터가 급격히 변하거나 noise가 큰 경우에는 과도하게 흔들릴 수 있다.

보간법 선택은 데이터의 성격에 따라 달라져야 한다.

Spline Interpolation

Example

다음과 같은 이산 데이터가 주어졌다고 하자.

$$(x, y) = (1, 2), (2, 4), (3, 3), (4, 5), (5, 7)$$

이 데이터는 선형적이지 않고 굴곡이 존재한다. 따라서 단순한 직선(linear interpolation)보다 더 부드러운 곡선으로 데이터 사이 값을 추정하고자 한다. 이때 **spline interpolation**을 사용하여 연속적이고 부드러운 함수 형태로 데이터를 연결한다.

```
clear; clc;
x = 1:5;                                % Define observed x-values
y = [2 4 3 5 7];                        % Define observed y-values
xq = 1:0.1:5;                           % Define query points
yq = interp1(x,y,xq,'spline');           % Perform spline interpolation
plot(x,y,'o',xq,yq,'LineWidth',1.5);    % Plot data and interpolation
xlabel('x');                             % Add x-axis label
ylabel('y');                             % Add y-axis label
legend('Data','Spline');                 % Add legend
```

```
clear; clc;                                % Clear workspace and command window
x = 0:1:10;                                % Define observed x-values
y = sin(x);                                % Define observed y-values
xq = 0:0.05:10;                             % Define dense query points
y_linear = interp1(x,y,xq,'linear'); % Compute linear interpolation
y_spline = interp1(x,y,xq,'spline'); % Compute spline interpolation
plot(x,y,'o',xq,y_linear,'--',xq,y_spline,'LineWidth',1.5); % Plot
legend('Data','Linear','Spline');           % Add legend
xlabel('x');                                % Add x-axis label
ylabel('y');                                % Add y-axis label
```

```
clear; clc; % Clear workspace and command window
x = 0:0.5:10; % Define time points
y = sin(x); % Generate clean signal
y([5 8 12]) = NaN; % Insert missing values
valid = ~isnan(y); % Find non-missing positions
y_fill = interp1(x(valid),y(valid),x,'linear'); % Fill missing data
plot(x,y,'o',x,y_fill,'LineWidth',1.5); % Plot original and filled data
legend('Observed','Interpolated'); % Add legend
xlabel('Time'); % Add x-axis label
ylabel('Value'); % Add y-axis label
```

실제 데이터에서는 결측값이 자주 발생한다.

- 센서 오류로 특정 시점의 데이터가 빠질 수 있다.
- 실험 과정에서 일부 측정이 누락될 수 있다.
- 시간 간격이 일정하지 않은 데이터도 발생할 수 있다.

보간법은 이러한 불완전한 데이터를 분석 가능한 형태로 복원하는 기본적인 방법이다.

심화 예제: Noisy Sensor Data 복원

```
clear; clc;                                % Clear workspace and command window
rng(1);                                    % Fix random seed
x = linspace(0,10,60);                     % Generate time points
true_y = sin(x) + 0.3*cos(3*x);            % Generate true signal
noise = 0.15*randn(size(x));               % Generate random noise
obs_y = true_y + noise;                    % Create noisy observations
miss = randperm(length(x),12);             % Select missing positions
obs_y(miss) = NaN;                         % Insert missing values
valid = ~isnan(obs_y);                     % Find valid observations
y_rec = interp1(x(valid),obs_y(valid),x,'spline'); % Recover signal
plot(x,true_y,'-',x,obs_y,'o',x,y_rec,'--','LineWidth',1.5); % Plot
legend('True','Observed','Recovered'); % Add legend
```

이 예제는 실제 센서 데이터 분석과 비슷한 구조를 갖는다.

- 참값은 부드러운 신호로 설정하였다.
- 관측값에는 정규분포 잡음을 추가하였다.
- 일부 관측값을 결측값으로 만든 뒤 spline 보간으로 복원하였다.

이 과정은 데이터 생성, noise modeling, missing data recovery를 모두 포함한다.

```
clear; clc; % Clear workspace and command window
x = 0:1:10; % Define sparse x-values
y = sin(x); % Define observed y-values
xq = 0:0.05:10; % Define dense query points
true_y = sin(xq); % Compute true function values
y_linear = interp1(x,y,xq,'linear'); % Linear interpolation
y_spline = interp1(x,y,xq,'spline'); % Spline interpolation
rmse_linear = sqrt(mean((true_y-y_linear).^2)); % Compute linear RMSE
rmse_spline = sqrt(mean((true_y-y_spline).^2)); % Compute spline RMSE
fprintf('Linear RMSE = %.4f\n',rmse_linear); % Print linear RMSE
fprintf('Spline RMSE = %.4f\n',rmse_spline); % Print spline RMSE
```

RMSE는 예측값과 실제값 사이의 차이를 측정하는 대표적인 지표이다.

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

- RMSE가 작을수록 복원 결과가 실제값에 가깝다.
- Linear와 spline 보간법을 정량적으로 비교할 수 있다.
- 시각적 비교와 수치적 비교를 함께 사용하는 것이 좋다.

Integrated Project

난수, 시뮬레이션, 보간법을 모두 결합하여 다음과 같은 순서로 진행해보자.

1. 참 신호를 생성한다.
2. 정규분포 난수로 noise를 추가한다.
3. 일부 관측값을 결측값으로 만든다.
4. 보간법으로 결측값을 복원한다.
5. RMSE로 복원 성능을 평가한다.

```
clear; clc; % Clear workspace and command window
rng(10); % Fix random seed
x = linspace(0,20,100); % Generate time points
true_y = sin(x) + 0.5*sin(2*x); % Generate true signal
obs_y = true_y + 0.2*randn(size(x)); % Add random noise
miss = randperm(length(x),20); % Select missing positions
obs_y(miss) = NaN; % Insert missing values
valid = ~isnan(obs_y); % Find observed positions
y_lin = interp1(x(valid),obs_y(valid),x,'linear'); % Linear recovery
y_spl = interp1(x(valid),obs_y(valid),x,'spline'); % Spline recovery
rmse_lin = sqrt(mean((true_y-y_lin).^2)); % Linear RMSE
rmse_spl = sqrt(mean((true_y-y_spl).^2)); % Spline RMSE
plot(x,true_y,x,obs_y,'o',x,y_lin,'--',x,y_spl,'-.','LineWidth',1.3); % Plot
legend('True','Observed','Linear','Spline'); % Add legend
```

이 프로젝트에서 중요한 점은 단순히 그래프를 그리는 것이 아니라 데이터 분석의 전체 흐름을 경험하는 것이다.

- 난수는 noise와 missing pattern을 생성하는 데 사용된다.
- 시뮬레이션은 다양한 결측 상황을 반복적으로 만들 수 있게 한다.
- 보간법은 손상된 데이터를 복원하는 방법을 제공한다.
- RMSE는 복원 결과를 객관적으로 비교하게 한다.

Practice Problems

실습문제 1-5

1. rand를 이용하여 0과 1 사이의 난수 500개를 생성하고 histogram을 그리시오.
2. 평균이 50, 표준편차가 8인 정규분포 난수 1000개를 생성하고 평균과 표준편차를 계산하시오.
3. 주사위를 2000번 던지는 시뮬레이션을 수행하고 각 눈의 빈도를 histogram으로 나타내시오.
4. 동전을 5000번 던지고 앞면의 누적 비율이 0.5에 가까워지는 과정을 그래프로 나타내시오.
5. Monte Carlo simulation을 이용하여 π 를 추정하시오.

실습문제 6-10

6. Monte Carlo 방법을 이용하여 $\int_0^1 x^3 dx$ 를 근사하시오.
7. 길이가 500인 random walk를 생성하고 그래프로 나타내시오.
8. $x = 0 : 1 : 10$, $y = \sin(x)$ 에 대해 linear interpolation과 spline interpolation을 비교하시오.
9. 일부 값이 NaN인 시간 데이터에서 interp1을 이용하여 결측값을 복원하시오.
10. noise가 있는 신호에서 임의로 20%의 데이터를 제거한 뒤 linear와 spline 보간의 RMSE를 비교하시오.

Code Answers

코드답안 1

```
clear; clc; % Clear workspace and command window
x = rand(1,500); % Generate 500 uniform random numbers
histogram(x); % Draw histogram
title('Uniform Random Numbers'); % Add title
xlabel('Value'); % Add x-axis label
ylabel('Frequency'); % Add y-axis label
```

```
clear; clc; % Clear workspace and command window
mu = 50; % Set mean
sigma = 8; % Set standard deviation
x = mu + sigma*randn(1,1000); % Generate normal random numbers
sample_mean = mean(x); % Compute sample mean
sample_std = std(x); % Compute sample standard deviation
fprintf('Mean = %.2f\n',sample_mean); % Print mean
fprintf('Std = %.2f\n',sample_std); % Print standard deviation
histogram(x); % Draw histogram
```

```
clear; clc;                                % Clear workspace and command window
dice = randi([1 6],1,2000);                % Simulate 2000 dice rolls
histogram(dice,1:7);                        % Draw histogram with dice bins
title('Dice Simulation');                  % Add title
xlabel('Dice Value');                       % Add x-axis label
ylabel('Frequency');                       % Add y-axis label
```

```
clear; clc; % Clear workspace and command window
n = 5000; % Set number of coin tosses
coin = rand(1,n) > 0.5; % Simulate coin tosses
p_head = cumsum(coin)./(1:n); % Compute cumulative proportion
plot(p_head,'LineWidth',1.5); % Plot cumulative proportion
yline(0.5,'--'); % Add reference line
xlabel('Number of Tosses'); % Add x-axis label
ylabel('Proportion of Heads'); % Add y-axis label
```

```
clear; clc; % Clear workspace and command window
n = 10000; % Set number of random points
x = rand(1,n); % Generate x-coordinates
y = rand(1,n); % Generate y-coordinates
inside = (x.^2 + y.^2) <= 1; % Check inside quarter circle
pi_est = 4*mean(inside); % Estimate pi
fprintf('pi estimate = %.4f\n',pi_est); % Print result
scatter(x,y,5,inside,'filled'); % Plot points
axis equal; % Use equal axis scale
```

```
clear; clc; % Clear workspace and command window
n = 10000; % Set number of samples
x = rand(1,n); % Generate samples from U(0,1)
f = x.^3; % Evaluate x^3
I_est = mean(f); % Estimate integral
fprintf('Integral estimate = %.4f\n',I_est); % Print estimate
fprintf('True value = %.4f\n',1/4); % Print true value
```

```
clear; clc; % Clear workspace and command window
n = 500; % Set number of steps
steps = randi([0 1],1,n); % Generate 0 or 1
steps = 2*steps - 1; % Convert to -1 or +1
walk = cumsum(steps); % Compute random walk
plot(walk,'LineWidth',1.5); % Plot random walk
xlabel('Step'); % Add x-axis label
ylabel('Position'); % Add y-axis label
```

```
clear; clc; % Clear workspace and command window
x = 0:1:10; % Define observed x-values
y = sin(x); % Define observed y-values
xq = 0:0.05:10; % Define query points
y_linear = interp1(x,y,xq,'linear'); % Linear interpolation
y_spline = interp1(x,y,xq,'spline'); % Spline interpolation
plot(x,y,'o',xq,y_linear,'--',xq,y_spline,'LineWidth',1.5); % Plot
legend('Data','Linear','Spline'); % Add legend
xlabel('x'); % Add x-axis label
ylabel('y'); % Add y-axis label
```

```
clear; clc; % Clear workspace and command window
x = 0:0.5:10; % Define time points
y = sin(x); % Generate signal
y([4 8 13 17]) = NaN; % Insert missing values
valid = ~isnan(y); % Find observed values
y_fill = interp1(x(valid),y(valid),x,'linear'); % Recover missing values
plot(x,y,'o',x,y_fill,'LineWidth',1.5); % Plot observed and filled data
legend('Observed','Interpolated'); % Add legend
xlabel('Time'); % Add x-axis label
ylabel('Value'); % Add y-axis label
```

코드답안 10

```
clear; clc; % Clear workspace and command window
rng(1); % Fix random seed
x = linspace(0,10,100); % Generate time points
true_y = sin(x) + 0.3*cos(2*x); % Generate true signal
obs_y = true_y + 0.15*randn(size(x)); % Add noise
miss = randperm(length(x),20); % Select 20 missing positions
obs_y(miss) = NaN; % Insert missing values
valid = ~isnan(obs_y); % Find observed positions
y_lin = interp1(x(valid),obs_y(valid),x,'linear'); % Linear interpolation
y_spl = interp1(x(valid),obs_y(valid),x,'spline'); % Spline interpolation
rmse_lin = sqrt(mean((true_y-y_lin).^2)); % Compute linear RMSE
rmse_spl = sqrt(mean((true_y-y_spl).^2)); % Compute spline RMSE
fprintf('Linear RMSE = %.4f\n',rmse_lin); % Print linear RMSE
fprintf('Spline RMSE = %.4f\n',rmse_spl); % Print spline RMSE
plot(x,true_y,x,obs_y,'o',x,y_lin,'--',x,y_spl,'-.','LineWidth',1.3); % Plot
legend('True','Observed','Linear','Spline'); % Add legend
```